

Functions

1. What is the main advantage of using functions in programming?
 - a. To make the program run faster
 - b. To make the program shorter
 - c. To allow for code reusability
 - d. To make the code less readable
2. Which of the following is a disadvantage of using functions in programming?
 - a. Increased code reusability
 - b. Easier debugging
 - c. Increased program complexity
 - d. Faster execution
3. What type of function is defined by the programmer?
 - a. Built-in function
 - b. Predefined function
 - c. User-defined function
 - d. System function
4. Which of the following is a type of user-defined function?
 - a. `sqrt()`
 - b. `main()`
 - c. `printf()`
 - d. None of the above
5. What is a built-in function?
 - a. A function that is defined by the user
 - b. A function that is part of the programming language library
 - c. A function that takes no arguments
 - d. A function that returns a string
6. What is a parameter in a function?
 - a. The return value of a function
 - b. The type of function
 - c. An argument passed to a function
 - d. The name of the function
7. What is the term for when a function calls itself?
 - a. Recursion
 - b. Iteration
 - c. Overloading
 - d. Encapsulation

8. What is the purpose of the return keyword in a function?
 - a. To indicate the function has finished
 - b. To send a value back to the function caller
 - c. To initialize the function's variables
 - d. To end the program
9. Which of the following can be passed to a function as an argument?
 - a. A constant value
 - b. A variable
 - c. A function
 - d. All of the above
10. What is the meaning of the term 'default argument' in a function?
 - a. An argument that is always required
 - b. An argument that has a predefined value if not specified by the caller
 - c. An argument passed by reference
 - d. An argument that cannot be modified
11. Which of the following is true about the scope of variables in a function?
 - a. Variables declared inside a function are global
 - b. Variables inside functions are accessible outside the function
 - c. Local variables cannot be accessed outside the function
 - d. Global variables cannot be accessed inside a function
12. What is the difference between a global variable and a local variable in a function?
 - a. Global variables are declared outside functions and can be accessed by any function, while local variables are confined to the function scope
 - b. Local variables are accessible globally
 - c. Global variables cannot be modified
 - d. Global variables are automatically passed as function arguments
13. What happens if the base case is not provided in a recursive function?
 - a. The function will return immediately
 - b. The program will crash
 - c. The recursion will never stop, resulting in an infinite loop
 - d. The program will return random results
14. What happens to memory usage when recursion is used?
 - a. Memory usage decreases
 - b. Memory usage stays constant
 - c. Memory usage increases as each function call adds a new frame to the call stack
 - d. Memory usage is unaffected

15. Which of the following can a function return?
- a. An integer
 - b. A string
 - c. A float
 - d. Any of the above
16. What is a common mistake when writing recursive functions?
- a. Not defining a base case
 - b. Using too many variables
 - c. Returning the wrong data type
 - d. Writing an infinite loop
17. Which of the following is not true about recursive functions?
- a. They are functions that call themselves
 - b. They require a base case to avoid infinite recursion
 - c. They are always more efficient than iterative solutions
 - d. They are useful for solving problems like factorial and Fibonacci series

18. What will the following code output?

```
#include <stdio.h>
```

```
int multiply(int a, int b) {  
    return a * b;  
}
```

```
int main() {  
    int result = multiply(3, 4);  
    printf("Result: %d\n", result);  
    return 0;  
}
```

- a. Result: 7
- b. Result: 12
- c. Result: 0
- d. Compilation error

19. What will the following code output?

```
#include <stdio.h>

void modifyValue(int x) {
    x = 50;
}

int main() {
    int num = 10;
    modifyValue(num);
    printf("Value of num: %d\n", num);
    return 0;
}
```

- a. Value of num: 10
- b. Value of num: 50
- c. Compilation error
- d. Undefined behavior

20. How many recursive calls are made for factorial(4)?

```
#include <stdio.h>
```

```
int factorial(int n) {
```

```
    if (n == 0)
```

```
        return 1; // Base case
```

```
    else
```

```
        return n * factorial(n - 1); // Recursive case
```

```
}
```

```
int main() {
```

```
    int num = 5;
```

```
    printf("Factorial of %d is %d\n", num, factorial(num));
```

```
    return 0;
```

```
}
```

- a. 4
- b. 5
- c. 3
- d. 6